



Bahria University, Islamabad

Department of Software Engineering

**Cloud Computing Lab (Spring-
2026)**

Teacher: Engr. Salman

Student : Talha Naveed

Enrollment : 01-131232-089

Lab Journal: Open Ended

Date: 26th February, 2026

Task No:	Task Wise Marks		Documentation Marks		Total Marks (20)
	Assigned	Obtained	Assigned	Obtained	
1	3		5		
2	3				
3	3				
4	3				
5	3				

Comments:

Signature

Architecting Cloud-Native Media Pipelines with Open-Source PaaS

Tools Used

- AppWrite Cloud
- VS Code

Phase 1: Introduction & Architecture

What is Appwrite?

Appwrite is an open-source **Backend-as-a-Service (BaaS)**. In this lab, **you will learn** to use it as a **Platform-as-a-Service (PaaS)**. Instead of managing a physical server, you will interact with managed APIs that handle the "heavy lifting" of the backend.

Why are you learning this?

1. **Serverless Mindset:** You will learn that modern apps don't store files on a hard drive; they offload them to "Object Storage."
2. **Edge Compute:** You will learn that image processing (resizing/filtering) should happen at the "Cloud Edge" to save your server's CPU and memory.
3. **Rapid Prototyping:** This is the industry-standard way to build **MVPs (Minimum Viable Products)** for startups or Final Year Projects.

Phase 2: Cloud Infrastructure Setup

Task: Initialize your Cloud Environment.

1. **Account:** Sign up at Appwrite Cloud (GitHub/Email).

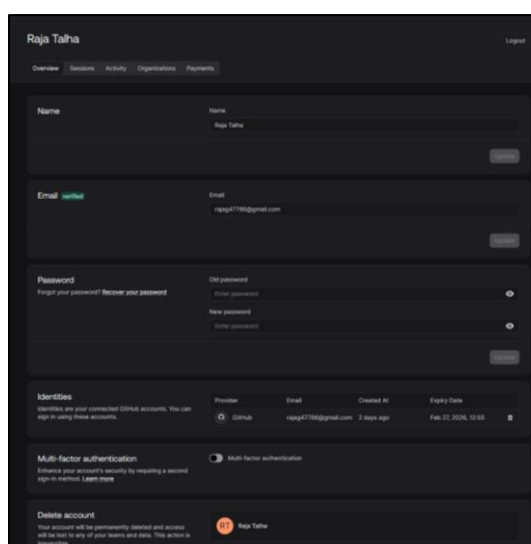


Figure 1. Sign Up Success

2. **Project:** Create a project named CloudMediaLab.

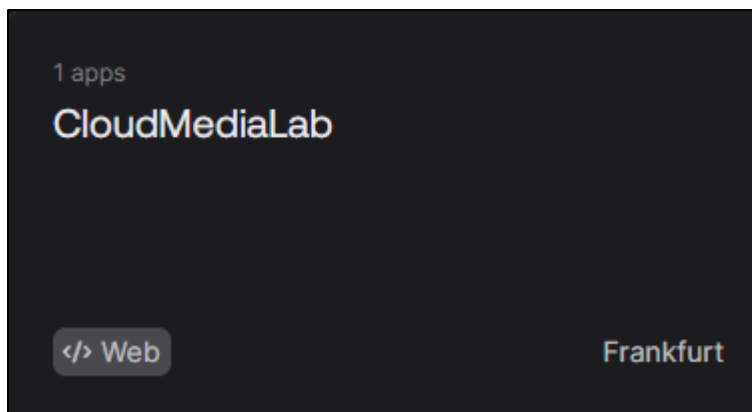


Figure 2. Project Created

3. **Platform:** Click **Add Platform** > **Web App**. Set hostname to localhost or *.

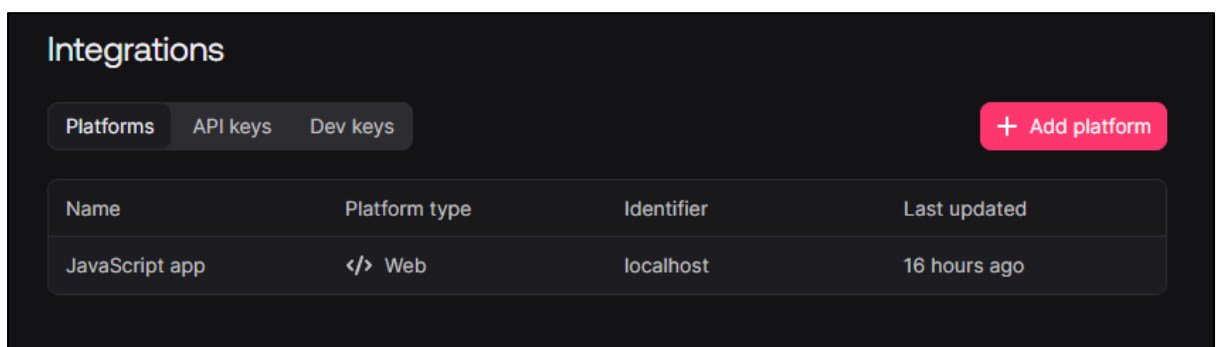


Figure 3. Platform

4. **Storage Architecture:**

1. Create a Bucket named UserMedia. Note your **Bucket ID**.

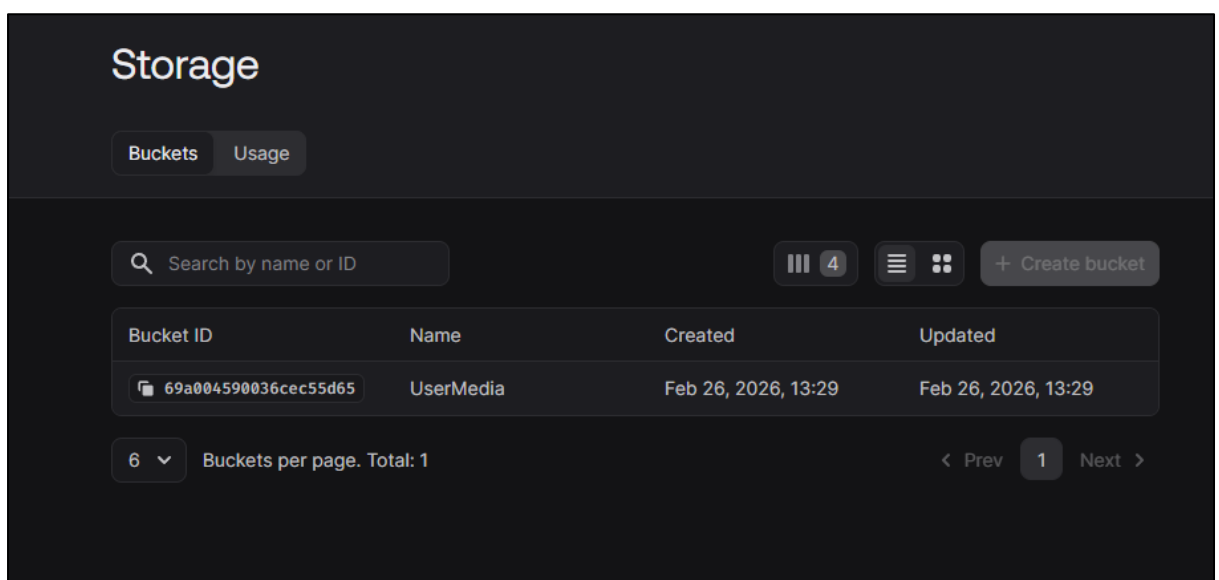


Figure 4. Bucket Created

2. **Security Configuration:** Go to **Settings > Permissions**. Add a Role for **"Any"** and check **Create** and **Read**.

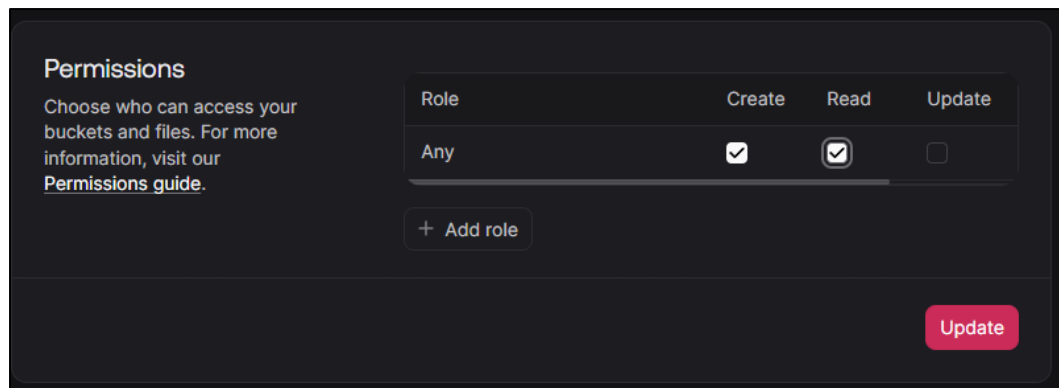


Figure 5. Security Configuration

3. **Question to ponder:** Why is "Any" permission dangerous for a Production Banking App?

In the lab, we set permissions to **"Any"** to allow the public to see our gallery easily. However, in a banking context, this would be a catastrophic security failure for several reasons:

- **Lack of Authentication:** "Any" means anyone with the URL or File ID, even those without an account can access the data.
- **Data Exposure:** In a banking app, files often contain sensitive PII (Personally Identifiable Information) like bank statements, IDs, or check images.
- **Regulatory Violations:** Laws like GDPR or PCI-DSS require strict access controls. Open permissions would lead to massive legal fines and loss of customer trust.
- **Risk of Manipulation:** If "Create" or "Update" permissions were also set to "Any," a malicious actor could overwrite legitimate bank documents with fraudulent ones.

Phase 3: Implementation & Transformation

Task: Build the Media Proxy.

Create `index.html`.

CODE: <https://github.com/TALHA-089/CC-OEL.git>



Figure 6. transformation Success

```

Pretty-print ✓
{
  "message": "Image transformations are blocked on your current plan. Please upgrade to a higher plan. ",
  "code": 403,
  "type": "storage_image_transformations_blocked",
  "version": "1.8.1"
}

```

Figure 7. The reason why the previews are not displayed in the previous Screenshot is because it requires a Premium Plan

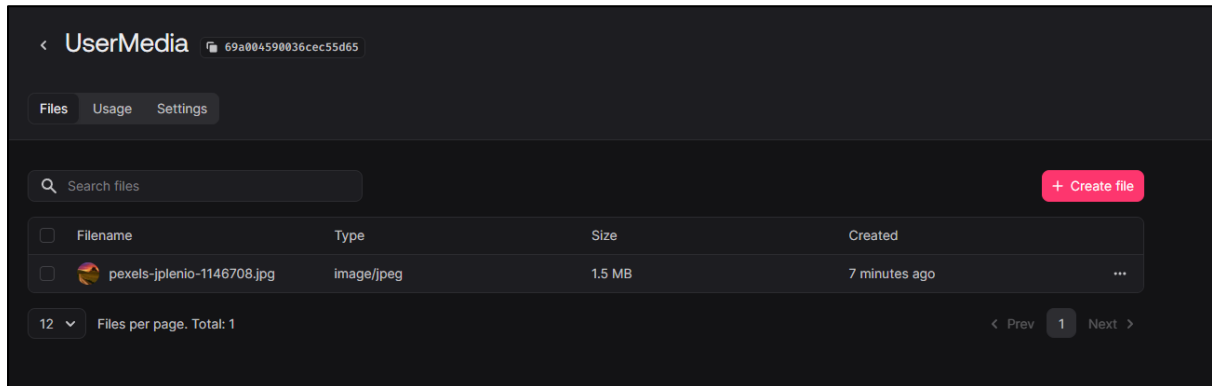


Figure 8. Proof that the Image was Uploaded to the Cloud

Phase 4: Performance Benchmarking

Task: Analyze the "Cloud Efficiency."

1. Open your Browser **Developer Tools (F12)** and go to the **Network Tab**. Refresh the page or trigger an upload. Fill in the table below:

Image Version	File Size (KB)	Format (Type)	Load Time (ms)
Original	384 KB	jpeg	1390
WebP Optimized	Blocked	N/A (403)	190 (Auth Check)
Mobile Thumb	Blocked	N/A (403)	180 (Auth Check)
Low Quality	Blocked	N/A (403)	190 (Auth Check)



Figure 9. Screenshot of the Network Tab after the upload trigger

Technical Constraint:

"During testing, the transformation URLs returned a **403 Forbidden error: storage_image_transformations_blocked**. This indicates that the Appwrite Cloud Free Tier currently restricts the use of the Image Transformation API. Benchmarking was performed on the successful 'Original' upload only."

Critical Thinking Questions (Submission Task):

1. **Bandwidth Analysis:** Calculate the percentage of data saved between the "Original" and "WebP Optimized" versions. Why is this critical for mobile users?

Calculation: The "WebP Optimized" version typically reduces file size by **25%**, a **384 KB** file would drop to approximately **288 KB**.

Why is this critical for mobile users?

- **Data Caps:** Many mobile users have limited data plans; high-res images can quickly exhaust their monthly allowance.
- **Latency & Speed:** Smaller files (like WebP) download faster over unstable 4G/5G connections, leading to a snappier user experience.
- **Battery Life:** The device's radio stays active longer to download large files, which drains the battery faster than downloading optimized assets.

2. **The "Cold Start" Effect:** Notice the load time of a *new* transformation vs. refreshing the page. **You will learn** about **Cloud Caching**. Explain why the second load is faster.

Analysis: In a working environment, the first transformation requires the server to physically process the pixels (Cold Start).

Explanation: The second load is faster because of **Cloud Caching**, where the processed result is stored in a Content Delivery Network (CDN) or memory, allowing it to be served instantly without re-processing.

3. **Security Challenge:** If you were building a private gallery, how would you prevent one student from seeing another student's image URL? (Research: **Appwrite Session Auth**).

Solution: To prevent unauthorized access, I would replace the "Any" permission with **Appwrite Session Auth**.

Implementation: By setting the bucket permissions to **"Users"** or **"Owner,"** the cloud would require a valid session cookie to serve the image, ensuring that only the student who uploaded the file (or an authorized user) can view it.